Instituto Superior de Engenharia de Coimbra
Instituto Politécnico de Coimbra

Licenciatura Engenharia Informática

Programação Orientada de Objetos

2025/2026

Meta 2

Realizado por:

João bernardo Libório Graça(2024128702) - a2024128702@isec.pt

Índice

1 INTRODUÇÂO III
2 ARQUITETURA E DECISÕES DE DESIGN III
2.1 ARQUITETURA GERAL E PADRÕES DE DESIGN III
2.2 ESTRUTURAS DE DADOS E GESTÃO DE MEMÓRIA IV
3 ESTRUTURA DO PROJETO EM CLASSES V
3.1 DIAGRAMA DE CLASSES UML(UNIFIED MODELING LANGUAGE) V
3.1 LISTA DE REQUISITOS VI
3.2 DESCRIÇÃO DAS CLASSES PRINCIPAIS VII
4 CONCLUSÃO VIII

1 INTRODUÇÂO

Este relatório acompanha a entrega final do trabalho prático de Programação Orientada a Objetos, descrevendo a implementação completa de um simulador de jardim em C++. O objetivo principal foi consolidar os conhecimentos sobre herança, polimorfismo, gestão dinâmica de memória e padrões de desenho.

Nesta segunda fase (Meta 2), o foco recaiu sobre a lógica de simulação: o ciclo de vida das plantas (crescimento, reprodução e morte), a interação do jardineiro com o meio através de ferramentas e a persistência de dados (gravar e carregar o estado do jogo). O projeto cumpre todos os requisitos propostos, oferecendo uma simulação robusta e interativa.

2 ARQUITETURA E DECISÕES DE DESIGN

A conceção da arquitetura do projeto foi guiada pela necessidade de criar um sistema robusto, organizado e de fácil manutenção, seguindo as boas práticas da engenharia de software.

2.1ARQUITETURA GERAL E PADRÕES DE DESIGN

A arquitetura do projeto é centralizada na classe Simulador, que atua como um “Maestro”. Esta classe é responsável por gerir o ciclo de vida dos objetos centrais (Jardim e Jardineiro) e por processar o input do utilizador.

Para a gestão de comandos, embora não fosse um requisito obrigatório, optei por implementar o Command Pattern. Esta decisão foi tomada para substituir uma potencial cadeia de if-else if por uma arquitetura mais flexível e alinhada com os princípios de POO. Cada comando do utilizador é encapsulado num objeto que herda de uma classe base abstrata **Comando**. O método interpretaComando do Simulador atua como uma Simple Factory, responsável por validar a sintaxe do input e construir o objeto de comando correto. O loop principal do simulador, no método iniciar(), apenas executa o comando de forma polimórfica, sem precisar de saber qual é. Esta abordagem, embora opcional, simplifica o código principal e facilita a adição de novos comandos no futuro (Princípio Aberto/Fechado).

Para modelar os diferentes tipos de plantas e ferramentas, recorreu-se a herança, tal como abordado na Ficha 4. Foram criadas classes base abstratas (Planta, Ferramenta) que

definem um "contrato" comum através de métodos virtuais puros (ex: atualizar()). As classes derivadas (Cato, Roseira, Regador, etc.) implementam este contrato, cada uma com o seu comportamento específico. Esta estrutura prepara o sistema para o uso de polimorfismo na Meta 2.

2.2ESTRUTURAS DE DADOS E GESTÃO DE MEMÓRIA

A escolha das estruturas de dados foi ditada pelos requisitos do enunciado.

Para o armazenamento do solo do jardim, e de forma a cumprir a restrição explícita do enunciado, a grelha foi implementada como um **array 2D alocado dinamicamente (`Posicao**`)**. A gestão da memória é assegurada no construtor da classe `Jardim` (com `new`) e no seu destrutor (com `delete[]`), garantindo a ausência de fugas de memória. A memória alocada para cada objeto `Planta` é, por sua vez, gerida pelo destrutor da classe `Posicao` que o contém.

Para o inventário de ferramentas do jardineiro, onde a restrição não se aplicava, optou-se por usar um `std::vector<Ferramenta*>`, por ser a estrutura de dados da biblioteca standard mais adequada para gerir uma coleção de tamanho variável.

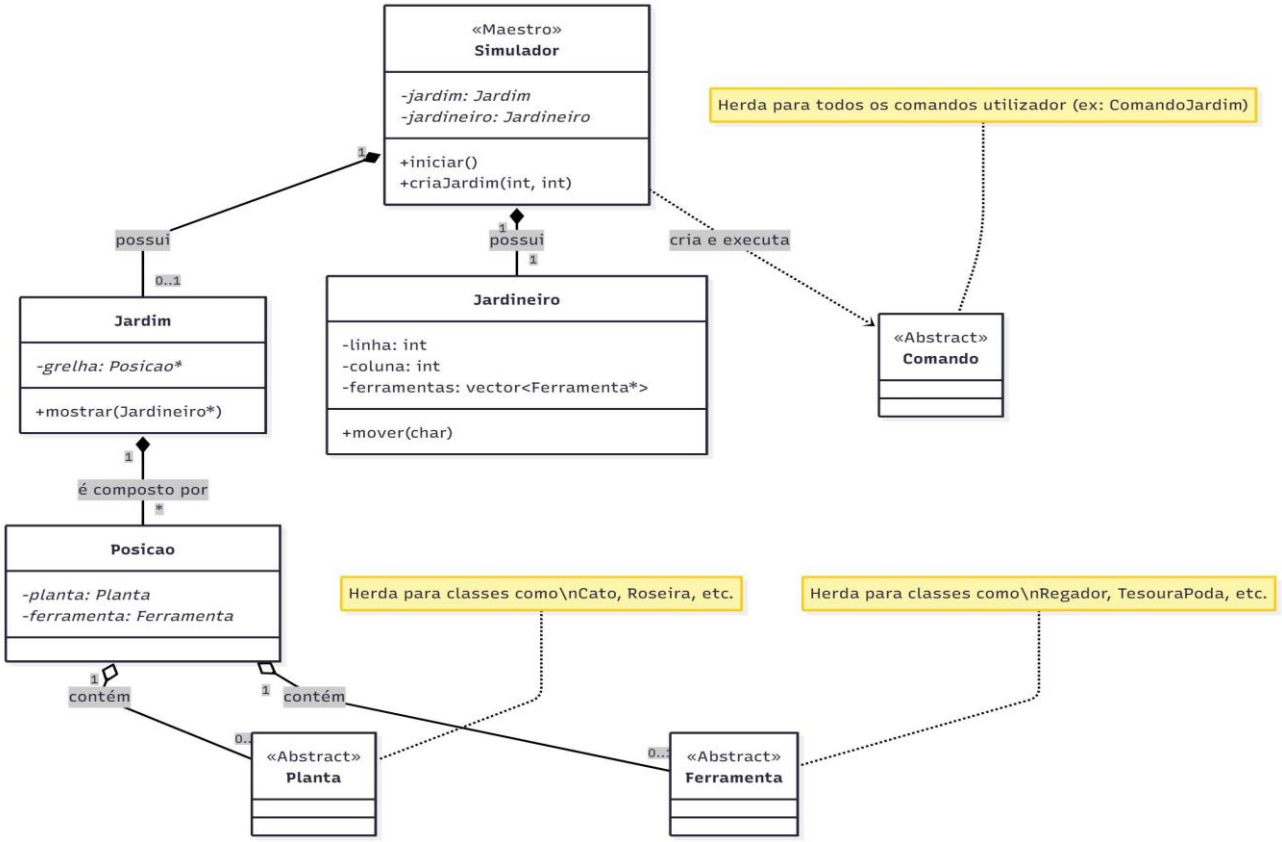
Para cumprir o requisito de não utilizar contentores da biblioteca standard (como std::vector) na estrutura do jardim, foi mantida a utilização de uma matriz dinâmica de ponteiros (Posicao** grelha).

Um desafio central nesta fase foi a implementação da funcionalidade de gravar e recupera. Para garantir que o estado do jardim fosse duplicado corretamente sem partilha de memória indevida (Deep Copy), foi implementado o padrão **Prototype** nas classes Planta e Ferramenta. Através do método virtual clone(), o jardim consegue criar cópias exatas de qualquer objeto (seja uma Roseira ou um Regador) sem precisar de conhecer o seu tipo concreto, garantindo a integridade dos dados guardados e evitando fugas de memória.

3 ESTRUTURA DO PROJETO EM CLASSES

A organização do projeto foi pensada para maximizar a coesão e minimizar o acoplamento, com cada classe a ter uma responsabilidade bem definida. As classes foram agrupadas em diretórios por funcionalidade (`plantas`, `ferramentas`, `comandos`) para uma melhor organização.

3.1DIAGRAMA DE CLASSES UML(UNIFIED MODELING LANGUAGE)



DIAGRAMASIMPLIFICADO

3.1 LISTA DE REQUISITOS

Categoria	Requisito	Estado	Observações
Geral	Simulação do Jardim e Grelha	✅ Implementado	Matriz dinâmica sem std::vector.
Entidades	Jardineiro (Movimento e Ações)	✅ Implementado	Respeita limites de ações por turno.
	Ferramentas (Regador, Adubo, Tesoura)	✅ Implementado	Inclui compra, uso e desgaste.
	Ferramenta Extra (Pá / Z)	✅ Implementado	PaDeTransplante implementada.
	Plantas (Cato, Roseira, Erva)	✅ Implementado	Ciclos de vida, morte e reprodução completos.
	Planta Exótica (FlorDeOrvalho)	✅ Implementado	Comportamento definido pelo grupo.
Comandos	Gestão (jardim, sair, fim)	✅ Implementado	Validação robusta.
	Movimento e Interação (entra, pega...)	✅ Implementado	
	Informação (larea, lsolo, lplantas...)	✅ Implementado	Mostra detalhes de plantas e ferramentas no chão.

Categoria	Requisito	Estado	Observações
	Persistência (grava, recupera, apaga)	✅ Implementado	Usa *Deep Copy* para salvaguardar o estado total.
	Execução de Scripts (executa)	✅ Implementado	Lê e processa ficheiros de texto.
Regras	Passagem do tempo (avanca)	✅ Implementado	Atualiza plantas e ferramentas corretamente.
	Aleatoriedade e "Magia"	✅ Implementado	Geração de terreno e respawn de ferramentas.

3.2 DESCRIÇÃO DAS CLASSES PRINCIPAIS

`Simulador`: Classe central que dá voz a toda a simulação.

Responsabilidades: Gerir o ciclo de vida do `Jardim` e `Jardineiro`, interpretar o input do utilizador através da "fábrica de comandos" (`interpretaComando`) e providenciar os serviços de lógica de jogo (ex: `tentaPlantar`).

`Jardim`: Representa o tabuleiro do jogo .

Responsabilidades: Gerir a memória e o acesso à grelha de `Posicao`, validar coordenadas (`isPosicaoValida`), executar a ação de plantar e desenhar o estado visual do simulador (`mostrar`), cumprindo o requisito de representação visual da Meta 1.

`Jardineiro`: Representa o avatar do utilizador no simulador.

Responsabilidades: Manter o seu estado (posição e se está no jardim) e executar as ações básicas de movimento.

`Comando` (e derivadas): Representa um pedido do utilizador (Command Pattern).

Responsabilidades: Cada classe derivada (ex: `ComandoEntra`) encapsula a validação semântica (se a ação faz sentido no estado atual do jogo) e a execução de uma única ação, delegando a lógica aos objetos especialistas.

`Planta` (e derivadas): Representa os seres vivos do jardim.

Responsabilidades: A classe base define a interface comum. Cada derivada está preparada para implementar o seu comportamento único na Meta 2.

`Posicao`: Representa um único "bocado de solo" na grelha do jardim.

Responsabilidades: Encapsular o estado de uma célula individual do jardim, incluindo os seus níveis de água e nutrientes. Através de ponteiros, a Posicao gere a presença de uma Planta ou de uma Ferramenta. É também responsável por gerir o ciclo de vida da `Planta` que contém, destruindo-a no seu destrutor para garantir a correta libertação de memória.

4 CONCLUSÃO

O simulador de jardim encontra-se totalmente funcional, cumprindo todos os objetivos pedagógicos e funcionais propostos. A arquitetura baseada em comandos e polimorfismo revelou-se eficaz, permitindo adicionar novas plantas e ferramentas sem alterar a lógica central do simulador.

A gestão manual de memória, embora desafiante, foi controlada através de destrutores virtuais e mecanismos de cópia profunda, assegurando que o programa corre sem fugas de memória (memory leaks) mesmo durante ciclos longos de simulação. O resultado final é um sistema extensível, estável e capaz de lidar com todas as regras complexas de interação biológica descritas no enunciado.